

# Vision-Based Lane Following and Corner Navigation Using a Duckiebot in a Real-World Duckietown Environment

1<sup>st</sup> Steffen Ullmann

*Electrical Engineering Department  
Florida Polytechnic University  
Lakeland, USA  
sullmann3121@floridapoly.edu*

2<sup>nd</sup> Omar Kerbouche

*Electrical Engineering Department  
Florida Polytechnic University  
Lakeland, USA  
okerbouche3273@floridapoly.edu*

3<sup>rd</sup> Joseph Higgins

*Electrical Engineering Department  
Florida Polytechnic University  
Lakeland, USA  
jhiggins8936@floridapoly.edu*

**Abstract**—This paper presents a vision-based lane-following system for autonomous navigation of a Duckiebot in a structured urban environment using only monocular camera feedback. The approach extracts two geometric features from each frame—the vanishing point and midpoint of lane boundaries—through HSV color segmentation, Canny edge detection, and Hough line transformation. A PID controller uses the filtered vanishing point coordinate to compute steering commands, simplifying the nonlinear dual-feature controller proposed by Akbari et al. [1]. The system is implemented as a modular ROS architecture with separate nodes for vision processing, control, and state management. Experimental validation on a physical Duckiebot demonstrates robust straight-line tracking even under temporary loss of the yellow lane boundary, achieved through ROI expansion and exponential moving average filtering. However, significant limitations emerge in corner scenarios where averaged line detection fails to represent curved geometry, causing erratic vanishing point estimates and unstable control. Additional challenges include sensitivity to large initial heading errors. The results suggest that while minimal Image-Based Visual Servoing (IBVS) architectures can achieve reliable performance in structured straight segments without localization or mapping, robust corner navigation requires more sophisticated feature extraction and potentially nonlinear or model-based control strategies. The complete implementation is publicly available on GitHub<sup>1</sup>.

**Index Terms**—Duckiebot, vision-based control, lane following, vanishing point detection, autonomous navigation.

## I. INTRODUCTION

Autonomous driving in urban settings is usually described as a dense mix of sensing, localization, and heavy mapping, the sort of pipeline that grows complicated very quickly. In this project, we went in almost the opposite direction. The idea was to explore how far a minimal, vision-only strategy could take a small robot, and whether a Duckiebot equipped with just a monocular camera could navigate a simplified city layout without depending on the usual stack of supporting systems. The overall goal was simply to make the robot drive on its own by following a lane, but with the constraint that it had to rely entirely on what appeared in its camera feed.

Our methods were based on the work of Akbari et al. [1]. The authors present a lightweight visual servoing scheme

built around two image features extracted directly from the camera frame: the lane’s vanishing point and a midpoint estimate between the lane markings. These two quantities feed into a non-linear control law that attempts to keep the robot centered and stable while also allowing it to negotiate turns. We followed this same approach, adapting and improving pieces of it to our own platform when needed.

This technique falls within the broader area of Image-Based Visual Servoing (IBVS), which has been applied repeatedly in corridor-following and lane-keeping systems. Methods that detect vanishing points or projective line features have been used to steer robots through structured spaces, and the approach here fits reasonably well within that lineage [2]–[4].

By relying solely on vision, the design avoids odometry drift, map building, and the entire machinery of SLAM. This is a major simplification, but it suits the constraints of Duckietown and illustrates how low-cost arrangements can still manage basic autonomous navigation. All of our testing was done on a real Duckiebot moving through a physical Duckietown layout rather than within a simulation.

The work took place over several weeks in the Embedded Control course. Beyond getting the system to function, the project provided hands-on experience with visual perception, closed-loop control, and the practicalities of running a robotic system in real time, each of which behaved somewhat differently once transitioned from code to hardware.

## II. MATERIALS AND METHODS

### A. Hardware Platform

The Duckiebot is built around a compact onboard micro-computer responsible for all perception and control tasks. A wide-angle front-facing camera provides the sole source of sensory information used for navigation. Locomotion is achieved through a differential-drive system powered by two DC motors. No additional sensors such as LiDAR, wheel encoders, or GPS are used in this setup. As a result, all navigation decisions rely exclusively on real-time visual cues obtained from the camera.

<sup>1</sup>[https://github.com/SteffEng-lab/duckiebot\\_lane\\_following.git](https://github.com/SteffEng-lab/duckiebot_lane_following.git)

## B. Software Architecture

The software stack follows standard ROS conventions, with each computational component implemented as an independent node. These nodes communicate through publisher–subscriber message passing, providing modularity and robustness.

At a high level, the camera node streams raw images to the vision-processing node, which extracts geometric features required for navigation and performs basic state detection such as corner approach and lane boundary visibility. These features and state flags are passed to the controller node, which implements both the PID control law and safety logic for enabling/disabling motor output based on the detected conditions. Finally, motor commands are forwarded to the wheel-control node, which drives the robot.

Unlike more complex architectures with dedicated state-machine nodes, the state transitions in this implementation are distributed between the vision node (corner detection, line visibility monitoring) and the controller node (output enable/disable, emergency stops). This simplified approach reduces inter-node communication overhead while maintaining clear separation between perception and control responsibilities.

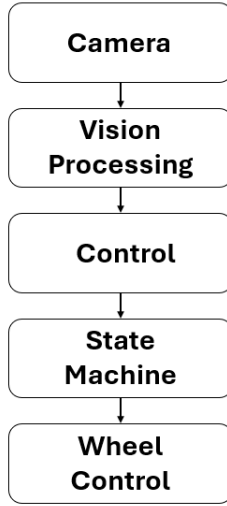


Fig. 1. Overall software architecture showing the flow from camera input to perception, control, state decisions, and wheel actuation. State machine logic is embedded within the vision and controller nodes rather than implemented as a separate component.

## C. Vision Processing

The vision-processing pipeline is mostly adapted from Akbari et al. [1], extracting the essential lane geometry needed for visual servoing. The process begins with acquiring raw camera frames from the Duckiebot’s forward-facing camera. An example of the original camera view with coordinate system orientation and the corresponding yellow/white lane extraction is shown in Fig. 2.

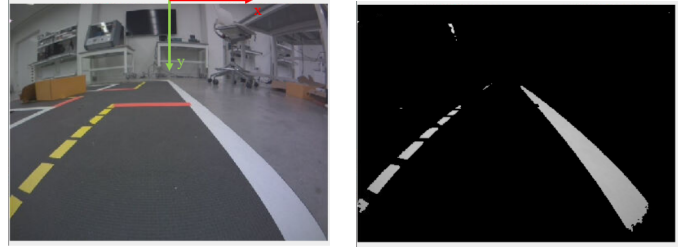


Fig. 2. Original camera view with coordinate system orientation (left) and corresponding binary extraction of yellow and white lane markings using HSV color filtering (right).

1) *Preprocessing and Color Segmentation*: Each incoming frame undergoes Gaussian blurring with a  $3 \times 3$  kernel to reduce high-frequency noise. The image is then converted from BGR to HSV color space, enabling robust color-based segmentation under varying lighting conditions.

Two binary masks are generated using the `cv2.inRange()` function:

- **White lane boundary**: HSV range  $[0, 0, 178] \leq (H, S, V) \leq [255, 51, 255]$ , isolating high-value, low-saturation pixels.
- **Yellow lane boundary**: HSV range  $[20, 76, 153] \leq (H, S, V) \leq [45, 255, 255]$ , capturing yellow hues with moderate-to-high saturation and value.

These masks are applied via bitwise AND operations to extract the colored lane regions, which are then converted to grayscale for subsequent edge detection.

2) *Edge Detection and Region of Interest*: The Canny edge detector is applied independently to the white and yellow grayscale images with thresholds of 100 and 200, producing two edge maps. To reduce computational cost and eliminate irrelevant features (e.g., sky, distant objects), a region of interest (ROI) mask is applied:

$$\text{ROI}(x, y) = \begin{cases} 1 & \text{if } y \geq h/3 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where  $h$  is the image height. This restricts processing to the lower two-thirds of the frame, where lane markings are most prominent.

3) *Line Detection via Hough Transform*: Lane boundaries are detected using the probabilistic Hough line transform in polar coordinates  $(\rho, \theta)$ , where  $\rho$  is the perpendicular distance from the origin to the line and  $\theta$  is the angle of the perpendicular. Different accumulator thresholds are used for each color due to their distinct visibility characteristics:

- **White lines**: threshold = 70 votes (stricter due to continuous shoulder line)
- **Yellow lines**: threshold = 40 votes (relaxed due to dashed centerline)

All detected line segments of the same color are averaged to produce a single representative line for each boundary:

$$\rho_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \rho_i, \quad (2)$$

$$\theta_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \theta_i, \quad (3)$$

where  $N$  is the number of detected segments.

4) *Feature Point Computation*: From the averaged white and yellow lines in polar form  $(\rho_w, \theta_w)$  and  $(\rho_y, \theta_y)$ , two geometric features are computed:

a) *Vanishing Point*: The intersection of the extended lane boundaries is found by converting to Cartesian form  $y = ax + b$  and solving:

$$a_w = -\frac{\cos \theta_w}{\sin \theta_w}, \quad b_w = \frac{\rho_w}{\sin \theta_w}, \quad (4)$$

$$a_y = -\frac{\cos \theta_y}{\sin \theta_y}, \quad b_y = \frac{\rho_y}{\sin \theta_y}, \quad (5)$$

$$x_v = \frac{b_w - b_y}{a_y - a_w}, \quad y_v = a_w x_v + b_w. \quad (6)$$

Division-by-zero checks are performed: if  $|\sin \theta| < 0.01$  (nearly horizontal lines) or  $|a_y - a_w| < 0.01$  (parallel lines), the intersection is discarded as invalid.

b) *Middle Point*: The midpoint between the intersections of each lane boundary with the bottom image row ( $y = h$ ) is computed:

$$x_{w,\text{bottom}} = \text{intersection of white line with } y = h, \quad (7)$$

$$x_{y,\text{bottom}} = \text{intersection of yellow line with } y = h, \quad (8)$$

$$x_m = \frac{x_{w,\text{bottom}} + x_{y,\text{bottom}}}{2}. \quad (9)$$

Both feature points are clamped to image boundaries and transformed to image-centered coordinates by subtracting  $w/2$ , where  $w$  is image width. Figure 3 illustrates the detected lines and feature points overlaid on the processed image.

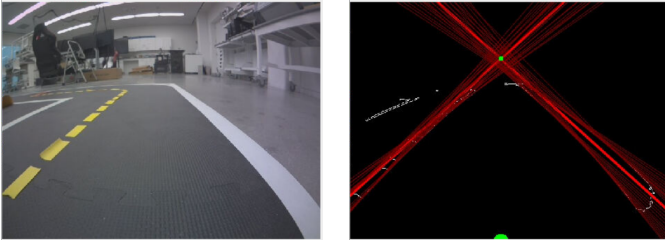


Fig. 3. Original camera view (left) and processed output (right) showing lane boundaries (white), individual Hough lines (thin red), average yellow and white guide line (thick red), vanishing point (upper green marker), and middle point (lower green marker).

5) *Lane Visibility and Corner Detection*: The system monitors lane boundary visibility by checking whether the Hough transform returns valid lines. Boolean flags `white_line_visible` and `yellow_line_visible`

are published to indicate detection status, enabling the controller to activate safety behaviors.

Corner detection is triggered when two conditions are simultaneously met:

- 1) The yellow (left) boundary is not detected, indicating a potential turn or intersection.
- 2) The white line angle relative to the horizontal axis satisfies  $0^\circ < \theta_w < 15^\circ$ , indicating the lane boundary has become nearly horizontal in the robot's field of view.

This conservative heuristic prevents false positives from temporary occlusions while reliably detecting approach to sharp turns.

These visual features—vanishing point  $x_v$ , middle point  $x_m$ , and visibility flags—are published via ROS topics at the camera frame rate ( $\sim 30$  Hz) and serve as the sole inputs to the downstream controller, allowing the system to operate without auxiliary sensing such as odometry or range measurements.

#### D. Control and State Machine

The control architecture differs from the nonlinear feature-based controller presented in [1] in two fundamental ways: (1) it uses only the vanishing point  $x_v$  as the control input, omitting the middle point  $x_m$ , and (2) it employs a linear PID controller instead of the nonlinear control law. This simplified approach was chosen to reduce computational complexity with less parameters to adjust while maintaining robust lane-following performance. Adjusting the control parameters was left mainly unspecified in [1] making this work hard to reproduce. Our PID controller however, can be tuned simply by trial and error and common tuning guidelines.

1) *PID-Based Angular Velocity Control*: The controller computes the robot's angular velocity  $\omega$  using a discrete-time PID control law applied to the vanishing point error. The vanishing point coordinate  $x_v$  is first filtered using an exponential moving average (EMA) to reduce measurement noise:

$$x_{v,\text{filtered}}(n) = \alpha \cdot x_v(n) + (1 - \alpha) \cdot x_{v,\text{filtered}}(n - 1), \quad (10)$$

where  $\alpha \in [0, 1]$  is the filter coefficient (set to 0.7 for responsive yet stable behavior).

The control error is defined relative to a target vanishing point position:

$$e(t) = x_{v,\text{target}} - x_{v,\text{filtered}}(t), \quad (11)$$

where  $x_{v,\text{target}} = -30$  pixels, representing a slight leftward offset to compensate for the camera's horizontal mounting offset to maintain straight motion.

The PID control law then computes the angular velocity as:

$$\omega(t) = k_p e(t) + k_i \int_0^t e(\tau) d\tau + k_d \frac{de(t)}{dt}, \quad (12)$$

which is implemented in discrete time as:

$$\omega(t) = k_p e(t) + k_i \sum_{j=0}^t e(j) \Delta t + k_d \frac{e(t) - e(t-1)}{\Delta t}, \quad (13)$$

where  $k_p = 0.001$ ,  $k_i = 0.0005$ , and  $k_d = 0.00001$  are the proportional, integral, and derivative gains, respectively. The integral term includes anti-windup limiting to prevent excessive accumulation:

$$\sum e(j)\Delta t \in \left[-\frac{\omega_{\max}}{k_i}, \frac{\omega_{\max}}{k_i}\right], \quad (14)$$

where  $\omega_{\max} = 2.0$  rad/s is the maximum allowed angular velocity.

Unlike the nonlinear controller in [1], which combines both vanishing point and middle point feedback through the control law

$$\omega = \frac{k_1}{k_1 k_3 + x_m x_v} (-k_2 k_1 v x_v - k_p x_m), \quad (15)$$

our simplified approach relies solely on vanishing point feedback. This simplification reduces the number of tunable parameters from four ( $k_1, k_2, k_3, k_p$ ) to three PID gains, and eliminates the potential singularity when  $k_1 k_3 + x_m x_v \approx 0$ .

2) *Differential Drive Velocity Conversion*: The computed angular velocity  $\omega$  and a constant forward velocity  $v = 0.005$  m/s are converted to individual wheel angular velocities using the differential drive kinematic model:

$$\omega_L = \frac{1}{R} \left( v - \frac{\omega L}{2} \right), \quad (16)$$

$$\omega_R = \frac{1}{R} \left( v + \frac{\omega L}{2} \right), \quad (17)$$

where  $R = 0.031$  m is the wheel radius,  $L = 0.094$  m is the wheelbase, and  $\omega_{L/R}$  are the left and right wheel angular velocities in rad/s. These are saturated to  $\pm 5.0$  rad/s before being published to the motor driver.

3) *State Machine and Safety Logic*: A hierarchical state machine supervises controller activation and safety-critical behaviors. The controller operates under three binary enable conditions:

- 1) **Output Enable** (`enable_output`): Controls whether motor commands are transmitted to the wheels. Initially disabled at startup, this flag can only be modified through ROS service calls (`/lane_controller/enable` and `/lane_controller/disable`), providing explicit user control over robot motion.
- 2) **Controller Enable** (`controller_enabled`): Determines whether the PID controller actively computes angular velocity or holds  $\omega = 0$ . This flag is automatically toggled by corner detection logic (see below).
- 3) **White Line Visibility** (`white_line_visible`): Acts as an emergency safety override. If the white lane boundary is lost for any reason, the system immediately stops all motion and disables output, preventing the robot from drifting out of the lane or into traffic.

The combined logic is implemented as:

```
if not enable_output:
    publish zero velocity
elif not white_line_visible:
```

```
    disable_callback() # emergency stop
    publish zero velocity
elif corner_detected:
    omega = 0.0, v = 0.0
    controller_enabled = False
    reset PID state
else:
    compute omega using PID
```

Unlike the scenario-driven transition system (STS) in [1], which uses red line detection to trigger predefined turn maneuvers with artificial guidelines, our implementation uses corner detection (loss of yellow lane boundary and exceeding white line angle threshold) to temporarily disable active control. This represents a conservative safety strategy: rather than executing autonomous turn maneuvers, the robot halts and awaits further instruction when lane geometry becomes ambiguous. The logic of this part will be replaced by autonomous corner following control in the future.

4) *PID State Management*: The PID controller state (integral accumulator, previous error, and timestamp) is reset to zero under three conditions: (1) when the enable service is called, (2) when the disable service is called, and (3) when a corner is detected. This prevents integral windup during periods of inactivity and ensures consistent behavior upon re-enabling. Additionally, a shutdown hook publishes zero velocity commands when the ROS node terminates, preventing uncontrolled motion during system shutdown.

5) *Comparison with Reference Controller*: Table I summarizes the key differences between our simplified PID-based controller and the nonlinear feature-based controller from [1].

This simplified control architecture demonstrates that robust lane following can be achieved using only vanishing point feedback with a classical PID controller, reducing implementation complexity while maintaining adequate performance for supervised or structured environments.

### III. RESULTS AND DISCUSSION

Three main scenarios were tested: Following a straight lane with corner at the end, analyzing the corner behavior in detail and starting at a steep angle to the lane.

#### A. Straight Line Performance and Missing Yellow Line Scenario

The first set of tests examined how the controller behaved on a straight lane, including a section where the yellow boundary vanished for a short period. The plots in Fig. 4 combine the target vanishing point, the measured value, and the angular velocity output, offering a compact view of the controller's behavior. Overall, the robot tracked the straight segment with little difficulty. Even with a fairly simple PID control structure, the estimated vanishing point stayed close to the target for most of the run. The target itself is not zero but roughly  $-30$  pixels, since the camera sits with a horizontal offset and requires that shift to maintain straight motion.

A notable moment occurs when the yellow line disappears halfway through the trajectory. Earlier ROI settings used only

TABLE I  
COMPARISON OF CONTROL APPROACHES

| Aspect           | Akbari et al.                                                                         | Our Work                                                                                                        |
|------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Control Inputs   | Vanishing point ( $x_v$ ) + middle point ( $x_m$ )                                    | Vanishing point ( $x_v$ ) only                                                                                  |
| Control Law      | Nonlinear feature-based: $\omega = f(x_v, x_m, v, k_1, k_2, k_3, k_p)$                | Linear PID: $\omega = k_p e + k_i \int e + k_d \dot{e}$                                                         |
| Turn Handling    | Autonomous turn execution using artificial guidelines triggered by red line detection | Conservative halt-and-wait behavior on corner detection. To be replaced in the future by autonomous control law |
| Parameters       | 4 nonlinear gains + scenario thresholds                                               | 3 PID gains + saturation limits                                                                                 |
| Singularity Risk | Potential division by zero when $k_1 k_3 + x_m x_v \approx 0$                         | None (linear controller)                                                                                        |

the lower half of the frame, which occasionally caused the yellow line to disappear in this region and the white line detection to become less stable. Increasing the ROI to cover two-thirds of the image improved the situation considerably. With more lane markings available, the detector rarely lost track of the lines, allowing it to detect the yellow lines even in far distance. This led to the robot continuously driving forward without reacting to the temporary absence of the yellow guideline.

Toward the end of the segment the vanishing point drops sharply into large negative values, indicating that the robot is approaching a left corner. Because the controller does not include corner-specific logic, this produces a sudden swing in the angular velocity and introduces instability. While the deviation might serve as a cue for corner detection, the signal is too noisy to trust by itself, since similar spikes occasionally appear during straight-line motion. The angular velocity plot in Fig. 4 supports this interpretation: calm behavior through the straight portion and far more oscillatory output near the corner.

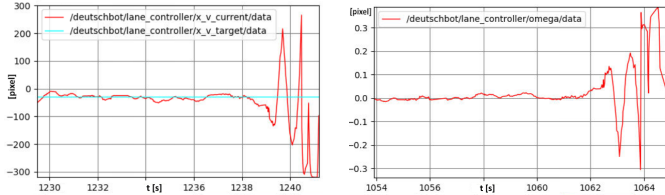


Fig. 4. Straight-line test: vanishing point target vs. measured value (left) and corresponding angular velocity output (right). The robot remains stable on the straight section and becomes more oscillatory near the corner

### B. Behavior Before Entering a Corner

A second experiment focused on the moments just before the robot reaches a left-hand corner. Enlarging the ROI exposes more of the lane geometry, and the white guideline becomes nearly tangent to the turning segment as the robot approaches it. This causes the lane to stretch across a greater portion of the frame, sometimes capturing the full bend rather than the short local segment we might expect. As both yellow and white lines start converging leftward, the vanishing point drifts in that direction as well. Only the last part of the plot in Fig. 5 matters here, since it captures the instant right before the trajectory bends.

Comparing the detection output with the raw camera image in Fig. 5 makes the interpretation clearer: the white line arcs toward the left, and the algorithm interprets this as a shift in the convergence point. Interestingly, this gradual shift could signal that the robot is nearing a corner. The idea is appealing, although, as the next results show, this cue does not remain reliable once the robot enters the curve.

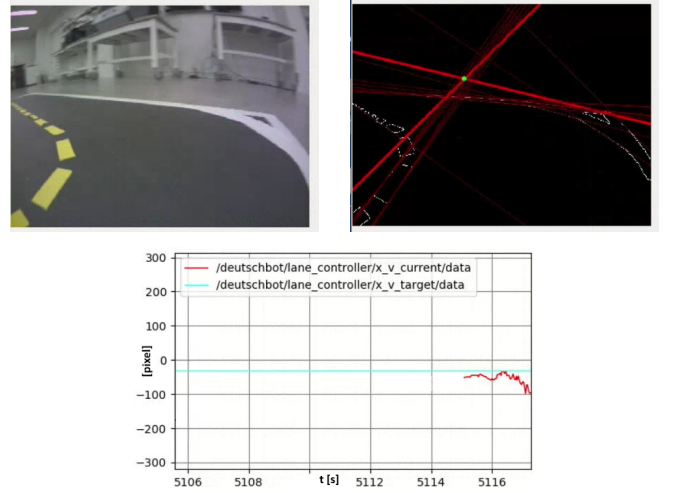


Fig. 5. Corner-approach test: original camera view (top left), extracted lane lines with vanishing point shifting left (top right), and the vanishing point trajectory over time (bottom).

### C. Behavior Inside the Corner

Once the robot is fully inside the corner, the detection process becomes far less dependable. The curved yellow boundary occupies much of the ROI and produces a dense collection of line fragments at multiple orientations. The averaging step collapses these fragments into a single representative line that often has little connection to the true local geometry. The result is a vanishing point that jumps erratically from frame to frame. The plotted values in Fig. 6 reflect this instability: large, abrupt swings rather than a consistent trend. With such behavior, the vanishing point becomes unsuitable for control, and the robot's motion becomes noticeably erratic.

This exposes a fundamental limitation of using globally averaged line segments for curved environments. As the geometry becomes more complex, the averaging process loses the ability to represent the robot's immediate direction of travel,

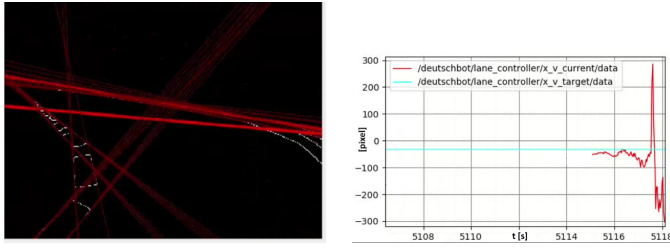


Fig. 6. Inside-corner test: processed image with dense, inconsistent lane detections (left) and vanishing point coordinate over time (right), showing strong frame-to-frame fluctuations.

instead capturing the global curvature of the entire visible lane arc.

Akbari et al. [1] circumvent this issue entirely by detecting red stop lines placed before each corner, triggering a transition to predefined artificial guidelines that guide the turn maneuver. However, our test environment more realistically represents unstructured scenarios where corners lack special markings. This necessitates vision-based corner detection and adaptive processing strategies.

One promising approach would leverage the gradual leftward drift of the vanishing point observed in Fig. 5 as a corner entry trigger. Upon detection, the system could dynamically reduce the ROI to cover only the immediate foreground ( $y \geq 2h/3$  instead of  $y \geq h/3$ ), restricting line detection to the local lane segment directly ahead of the robot. This would prevent the averaging algorithm from incorporating distant curved geometry while maintaining sufficient information for steering control.

#### D. Starting at a Steep Angle Relative to the Lane

A final experiment placed the robot at a large initial angle relative to the lane. The first camera frame in Fig. 7 makes the misalignment obvious, and the controller reacts strongly. The vanishing point oscillates as the robot tries to realign itself. Because the PID controller is not tuned for such large initial errors, it overshoots repeatedly, and the heading angle fluctuates. The angular velocity plot shows the same pattern, alternating between large corrective commands. The robot eventually stabilizes, but the process is slow and highly oscillatory, suggesting that additional damping or an improved control law would help in such scenarios.

#### IV. CONCLUSION

This work demonstrates that vision-based lane following using only vanishing point feedback and a simple PID controller can achieve robust performance on straight segments, even under temporary loss of lane boundaries. Key improvements over initial implementations include ROI expansion to two-thirds of frame height and exponential moving average filtering of the vanishing point, both of which significantly enhanced detection stability and noise rejection.

However, the approach reveals fundamental limitations when applied to curved trajectories. The global line averaging strategy, effective for straight geometry, fails to represent

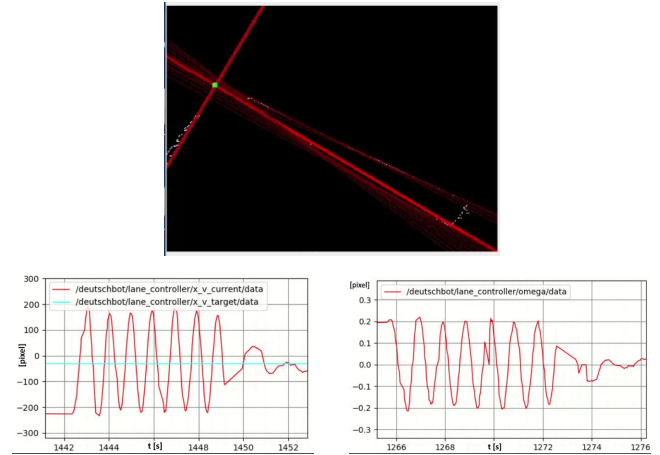


Fig. 7. Large initial misalignment test: starting camera view (top), vanishing point target vs. actual value (bottom left), and angular velocity control output (bottom right).

curved lane boundaries, producing erratic vanishing point estimates that render the controller unstable. Additionally, the PID gains tuned for nominal conditions exhibit poor transient response under large initial heading errors.

These findings suggest clear directions for future work. Corner navigation requires either adaptive ROI strategies that isolate relevant geometric segments or explicit curve-fitting methods that replace line averaging. The gradual leftward drift of the vanishing point observed before corner entry could serve as a reliable trigger for mode switching if combined with yellow line visibility monitoring. Model-predictive or nonlinear control strategies may offer improved disturbance rejection compared to the current linear PID structure.

Despite these limitations, the system successfully validates the core IBVS concept: autonomous navigation in structured environments is achievable using minimal sensing and computational resources. The complete open-source implementation provides a foundation for incremental enhancement and serves as an educational platform for hands-on robotics development.

#### REFERENCES

- [1] S. Akbari, N. Akbari, G. Oriolo, and S. Galeani, “Visual Urban Navigation for Mobile Robots: Implementation in the Duckietown Environment,” in *Proc. 2025 International Conference on Control, Automation and Diagnosis (ICCAD)*, pp. 1–6, 2025, doi: 10.1109/ICCAD64771.2025.11099311.
- [2] F. Pasteau, V. Narayanan, M. Babel, and F. Chaumette, “A visual servoing approach for autonomous corridor following and doorway passing in a wheelchair,” *Robotics and Autonomous Systems*, vol. 75, pp. 28–40, 2016.
- [3] R. F. Vassallo, H. J. Schneebeli, and J. Santos-Victor, “Visual servoing and appearance for navigation,” *Robotics and Autonomous Systems*, vol. 31, no. 1, pp. 87–97, 2000.
- [4] C. Zhong, Q. Kong, K. Wang et al., “Central Dioptric Line Image-Based Visual Servoing for Nonholonomic Mobile Robot Corridor-Following and Doorway-Passing,” *Actuators*, vol. 14, no. 4, p. 183, 2025.